

MCLAREN STRATEGIC SOLUTIONS
Ohio Medicaid Fiscal Intermediary — CEF Platform

End-to-End Capitation

Current-State Technical Architecture and Engineering Document

Document ID	CEF-CAP-TAD-001
Revision	rev 0.1 — DRAFT
Date	
Author (Solution Architect)	
Reviewer(s)	
Approver	
Engagement	IV&V Architecture Assessment — Contract 0A1286 / SOW 31066
Pillar	P1 — Critical Process Parallelism & CapRecon
Structure	§1 Architecture (~30%) · §2 Engineering (~70%)
Modelling standard	C4 views in the body · ArchiMate 4.0 (C260) model in Appendix B
Classification	MSS / Gainwell Confidential — not for State distribution without review
Status	TEMPLATE — no content authored

Delete the “TEMPLATE” status and set a revision number before circulating. Revisions increment 0.1 in draft, 1.0 at first issue. Do not renumber the document ID.

How to use this template

This page is scaffolding. Delete it before issue.

- The document has two parts. §1 Architecture states what the structure is; §2 Engineering states how it is actually built, at file, object and host grain. §0.9 gives the test for which part a statement belongs in. Roughly 30% of the written volume belongs in §1 and 70% in §2 — if §1 is growing past that, detail is leaking upward.
- Every architecture layer in §1 has exactly one matching engineering section in §2. The pairing is fixed: 1.3↔2.2, 1.4↔2.3, 1.5↔2.4, 1.6↔2.5 in Wave 1, and C.1↔C.2, C.3↔C.4 in Wave 2. A layer described with no evidence against it is an assertion, not a finding.
- Gold-barred italic text is instruction to the author. Delete each block once its section is written.
- Every table is empty by design. Add rows; do not change column headers without agreement, because downstream extracts key off them.
- Every substantive statement carries an evidence tag (§0.1). A section with untagged claims is not reviewable and will be returned.
- Every layer this document covers is listed in §0.8. A behavioural claim anywhere must resolve to a code citation, a runtime observation, or a named source.
- Where a fact already exists in an issued instrument, cite the instrument ID and do not restate the figure. Restated figures go stale independently and the corpus then disagrees with itself.
- Sections you cannot complete are marked NOT ASSESSED with a reason — never left blank and never silently dropped.
- This document describes what exists today. Target-state options, remediation designs and recommendations belong in the SOW and decision instruments. If a sentence contains "should", "could" or "we recommend", it is in the wrong document.

Section completion tracker

§	Section	Owner	Evidence plane	Status	Target date
2.4	<i>Data engineering</i>		<i>Runtime</i>	<i>Drafting</i>	

Contents

Page numbers are as issued in this blank template and will move once content is written. Headings use the built-in Word heading styles, so the list can be replaced at any point with a live field: References → Table of Contents → Custom, levels 1–3.

How to use this template	2
Section completion tracker	2
0. Document control and conventions	7
0.1 Evidence and confidence tags	7
0.2 Evidence planes	7
0.3 Data-handling constraint	7
0.4 Revision history	7
0.5 Distribution and review	8
0.6 Related instruments	8
0.7 Modelling conventions — C4	8
0.8 Layer coverage map	9
0.9 Architecture or engineering — which part does it belong in?	10
0.10 Using C4 to populate this document	10
0.10.1 Which level goes where	10
0.10.2 Working order	11
0.10.3 Which engineering section owns which C3	11
0.10.4 The traceability chain	12
0.10.5 Rules and common failure modes	12
0.10.6 Decomposition tracker	12
0.11 Delivery waves	13
1. Architecture	14
1.1 Scope and system boundary	14
1.1.1 In scope	14
1.1.2 Out of scope	14
1.1.3 Assumptions	15
1.2 End-to-end capitation architecture	15
1.2.1 Stage catalogue	15
1.2.2 Actors and responsibilities	15
1.2.3 Cycle types	16
1.3 Application architecture	16
1.3.1 Structure and responsibilities	16
1.3.2 Execution and orchestration model	16
1.3.3 COTS and customisation boundary	17
1.3.4 Decomposition posture	17
1.3.5 Architectural decisions as built	18
1.4 Integration architecture	18
1.4.1 Integration boundaries and counterparties	18
1.4.2 Integration styles in use	18
1.4.3 Managed-care lane	19
1.4.4 Architectural decisions as built	19
1.5 Data architecture	19
1.5.1 Stores and ownership	20
1.5.2 Bounded contexts and coupling	20
1.5.3 Data flow and lineage overview	20
1.5.4 Reference and rate data model	20
1.5.5 Architectural decisions as built	21
1.6 Infrastructure architecture	21
1.6.1 Topology and environments	21
1.6.2 Release lanes	21
1.6.3 Availability and recovery posture	22
1.6.4 Platform lifecycle position	22
1.6.5 Architectural decisions as built	22

1.7 Cross-cutting constraints and characteristics	23
1.7.1 Constraints	23
1.7.2 Architectural characteristics, as observed	23
1.8 Architecture views (C4 Levels 1–2)	23
1.8.1 Modelling scope and system boundary	23
1.8.2 View index.....	24
1.8.3 Views.....	24
1.8.4 View consistency check	24
2. Engineering.....	26
2.1 Engineering conventions.....	26
2.1.1 Code citation standard	26
2.1.2 Enumeration discipline.....	26
2.1.3 Environments and access used	26
2.1.4 Repository inventory	27
2.2 Application engineering.....	27
2.2.1 Component inventory	27
2.2.2 Entry points and scheduling	28
2.2.3 Configuration inventory	28
2.2.4 Stage execution detail.....	29
2.2.5 Control flow of the voucher / payment write path (C4 Level 4)	30
2.2.6 Transaction and commit semantics.....	30
2.2.7 Idempotency and restart behaviour.....	31
2.2.8 Error handling, retry and logging	31
2.2.9 COTS modification register.....	31
2.2.10 Build, release and artefact provenance	32
2.2.11 Static analysis coverage and blind spots.....	32
2.2.12 Component views (C4 Level 3) — application containers.....	33
2.2.13 Application defects and gaps	33
2.3 Integration engineering	34
2.3.1 Interface implementation inventory	34
2.3.2 Schemas, specifications and companion guides	34
2.3.3 Mapping and transformation objects.....	35
2.3.4 Transfer mechanics and landing zones	35
2.3.5 Acknowledgement, rejection and resubmission	36
2.3.6 Duplicate and replay handling	36
2.3.7 Cut-off enforcement and batching	37
2.3.8 Integration verification evidence.....	37
2.3.9 Component views (C4 Level 3) — integration containers	37
2.3.10 Integration defects and gaps	38
2.4 Data engineering	38
2.4.1 Instances and databases	39
2.4.2 Table inventory	39
2.4.3 Database code objects	40
2.4.4 Cross-database and linked-server dependencies.....	40
2.4.5 ETL and package inventory	41
2.4.6 Lineage	41
2.4.7 Reference and rate data loading	42
2.4.8 Indexing, contention and physical design.....	42
2.4.9 Data quality controls.....	42
2.4.10 Reconciliation and control engineering.....	43
2.4.11 Financial posting and downstream reporting	43
2.4.12 Retention, history and archival	44
2.4.13 Component views (C4 Level 3) — data containers	44
2.4.14 Data defects and gaps	45
2.5 Infrastructure and operations engineering.....	45
2.5.1 Host inventory	45
2.5.2 Deployment mapping	46
2.5.3 Job scheduling and orchestration	46
2.5.4 Observability and agent coverage.....	47
2.5.5 Backup, availability and recovery configuration	47
2.5.6 Lifecycle and end-of-support position	47
2.5.7 Run procedure	48

2.5.8 Manual touchpoints	48
2.5.9 Failure modes and restart procedures	49
2.5.10 Incident history	49
2.5.11 Support model	50
2.5.12 Component views (C4 Level 3) — operational containers	50
2.5.13 Infrastructure defects and gaps	51
3. Consolidated findings, defects and risks	52
3.1 Confirmed defects	52
3.2 Architectural constraints and structural issues	52
3.3 Risks	52
3.4 Retracted and refuted items	53
4. Open questions	54
5. Traceability	55
Appendix A — Business context	56
A.1 What capitation means on this platform	56
A.2 Business capabilities exercised	56
A.3 Contractual, SLA and regulatory drivers	56
A.4 Payment calendar and cycle	56
A.5 Volumes and materiality	57
A.6 Actors and responsibilities	57
Appendix B — ArchiMate 4.0 model	58
B.1 Standard and authority	58
B.2 Element mapping	58
B.3 Layer views	58
B.4 Cross-notation consistency check	59
B.5 Enterprise corpus alignment	59
Appendix C — Wave 2: Performance and Security	61
C.1 Performance architecture	61
C.1.1 Workload model	61
C.1.2 Scaling and serialisation points	61
C.1.3 Batch window and timing constraints	62
C.1.4 Performance targets applying to this path	62
C.1.5 Architectural decisions as built	62
C.2 Performance engineering	63
C.2.1 Measurement method and instrumentation	63
C.2.2 Observed volumes	63
C.2.3 Stage-level performance baseline	63
C.2.4 Bottleneck analysis	64
C.2.5 Query and plan-level findings	64
C.2.6 Critical path and window utilisation	65
C.2.7 Capacity and scaling limits	65
C.2.8 Load and stress test evidence	65
C.2.9 Position against stated targets	66
C.2.10 Performance defects and gaps	66
C.3 Security architecture	66
C.3.1 Trust boundaries	66
C.3.2 Identity and authorisation model	67
C.3.3 Data classification and PHI exposure	67
C.3.4 Access-model constraints	67
C.3.5 Architectural decisions as built	67
C.4 Security engineering	68
C.4.1 Identity and service account inventory	68
C.4.2 Authentication and authorisation implementation	68
C.4.3 Secrets and credential handling	69
C.4.4 Encryption in transit and at rest	69
C.4.5 PHI data flow and exposure points	69

C.4.6 Audit logging implementation.....	70
C.4.7 Access enforcement and segregation of duties.....	70
C.4.8 Patch and vulnerability position	70
C.4.9 Security defects and gaps.....	71
Appendix D — Evidence log.....	73
Appendix E — Source artefact inventory	74
Appendix F — Interview and walkthrough log	75
Appendix G — Glossary and abbreviations	76
Appendix H — Review and sign-off.....	77

0. Document control and conventions

0.1 Evidence and confidence tags

Mandatory. Every finding, figure and behavioural claim in this document carries exactly one tag. Untagged text is treated as unsupported.

Tag	Meaning	Citable externally?
CONFIRMED	Supported by file:line at a pinned revision, a named runtime query result, or a named source document.	Yes
INFERRED	Reasoned from evidence but not directly observed.	No — internal only
SME GATE	Requires confirmation from a named Gainwell SME. Record the name and the question.	No — internal only
EXTERNAL	Operating context sourced outside the platform (contract, regulation, vendor statement).	Yes, with source
REFUTED	Previously asserted, since disproved. Retained with the basis for retraction.	Yes, as a retraction

Rule of precedence: where the code plane and the runtime plane disagree, the runtime plane wins and the code-plane reading is recorded as REFUTED with the contradicting evidence.

0.2 Evidence planes

Code plane	
Runtime plane	
Both planes agree (fully CONFIRMED)	
Single-plane items outstanding	

Name the artefacts behind each plane: repository and commit for the code plane; query, host, database and run date for the runtime plane.

0.3 Data-handling constraint

State the Attachment Eight position as it applies to this document: which sections rest on production data, who executed those queries, and confirmation that no offshore personnel touched live Ohio production data. Name the onshore execution path.

Sections resting on production data	
Onshore executor	
De-identification applied	
Offshore-safe sections	

0.4 Revision history

Rev	Date	Author	Change	Sections affected

Rev	Date	Author	Change	Sections affected

0.5 Distribution and review

Name	Organisation	Role	Review / Approve / Inform	Date

0.6 Related instruments

List issued documents this one depends on or supersedes. Cite by ID and revision. Do not duplicate their contents here.

Instrument ID	Title	Rev	Relationship to this document
CEF-ENT-INT-002	Integration catalogue crosswalk	—	Source of interface identifiers used in §1.4 and §2.3

0.7 Modelling conventions — C4

Architecture views in §1.8 are C4. The formal ArchiMate 4.0 model behind them, and the mapping between the two, are in Appendix B. The two notations answer different questions and are never mixed on one canvas.

Question the reader is asking	Notation used	Section
What is in scope, who uses it, and what it touches outside the boundary	C4 L1 — System Context	§1.8 — Architecture
What separately deployable or executable units exist, and what data stores	C4 L2 — Container	§1.8 — Architecture
What happens, in what order, across containers during a cycle	C4 Dynamic view (container grain)	§1.8 — Architecture

Question the reader is asking	Notation used	Section
Where the containers run	C4 Deployment view (container grain)	§1.8 — Architecture
What sits inside a container and what each part is responsible for	C4 L3 — Component	§2.2, §2.3, §2.4, §2.5 — Engineering
What the code does on the write path	C4 L4 — Code	§2.2.5 — Engineering
The formal model behind these views, and corpus consistency	ArchiMate 4.0 (C260)	Appendix B

C4 has no formal metamodel and no defined relationship types; ArchiMate 4.0 has both. The default is that the model in Appendix B is authoritative and the C4 views are derived from it. Maintaining both by hand is not an option: they will disagree within one revision and the reader has no way to tell which is wrong.

Authoritative model	
Container boundary rule (instance / database / schema)	
Treatment of shared instances and cross-database coupling	
C4 element ID convention	<i>CAP-C4-PER-<i>nn</i> · SYS-<i>nn</i> · CON-<i>nn</i> · CMP-<i>nn</i> · NOD-<i>nn</i></i>
Authoring tool and model file location	

Every diagram issued satisfies all of the following before review. A diagram failing any line is returned, not annotated.

- *The title states the diagram type and the scope, so the reader never infers either from the shapes.*
- *A legend explains every shape, colour, border and arrow style used.*
- *Every element carries a name, an explicit type, a one-line responsibility, and its technology where the level calls for it.*
- *Every relationship is directional, labelled with an active phrase, and names the protocol or mechanism wherever it crosses a process boundary.*
- *Abstraction level is consistent: no components on a container diagram, no deployment detail on a logical view.*
- *Element names are identical across all levels and match the inventories in §2.*
- *The diagram carries an evidence tag. A view assembled partly from inference says so on its face.*

0.8 Layer coverage map

What this document claims to cover, and where. Each architecture layer has exactly one matching engineering section. A layer marked as covered but left empty at issue is an omission; a layer deliberately out of scope is recorded here with the reason.

Layer	Architecture (§1)	Engineering (§2)	Evidence plane
Operating context	Appendix A	Appendix A	External
Process / behaviour	§1.2	distributed across §2.2–§2.5	Runtime + interview
Application	§1.3	§2.2	Code + runtime
Integration	§1.4	§2.3	Code + runtime
Data	§1.5	§2.4	Code + runtime
Infrastructure and operations	§1.6	§2.5	Runtime + CSPM

Layer	Architecture (§1)	Engineering (§2)	Evidence plane
Performance — Wave 2	Appendix C.1	Appendix C.2	Runtime
Security — Wave 2	Appendix C.3	Appendix C.4	Code + runtime
Views	§1.8 — C4 L1, L2	C4 L3 in §2.2/2.3/2.4/2.5 · L4 in §2.2.5	Derived
Evidence	—	Appendices C–E	All

Layers deliberately out of scope	
Reason	
Layers covered but not yet populated at this revision	

0.9 Architecture or engineering — which part does it belong in?

Apply this test to every statement. The two parts fail in opposite directions: §1 fails by accumulating detail until nobody can see the structure, §2 fails by asserting behaviour without producing the artefact that proves it.

If the statement is...	It belongs in	Because
A structure, boundary, responsibility or dependency	§1	It describes shape, and shape is what a reader needs first
A named file, object, host, account, job or config key	§2	It is an artefact and must carry a citation
A count of anything	§2	Counts are enumerated, and the enumeration command is recorded with them
A decision visible in the as-built structure	§1	Record it as built, with the constraint that forced it
A behaviour observed at runtime	§2	It needs the query, host and date that observed it
A measured number with a unit	§2	Measurements carry a method and an observation window
A defect, gap or unreconciled difference	§2, then §3	Evidenced in the layer that owns it, consolidated in the register
A constraint imposed from outside the platform	§1.7	It shapes every layer and should be stated once
A contractual driver, volume or calendar fact	Appendix A	Operating context, cited from the body rather than restated

0.10 Using C4 to populate this document

Author guidance. C4 is the mechanism that keeps §1 and §2 in step: the levels split across the two parts on exactly the same line the parts themselves split on. C1 and C2 answer "what is this and what are its pieces" — that is structure, and it belongs in §1. C3 and C4 answer "what is inside a piece and what does it actually do" — that is evidence, and it belongs in §2. Populate the levels in order; each one constrains the next.

0.10.1 Which level goes where

C4 level	Question it answers	Part	Section	Evidence grain required
C1 — System Context	Boundary, users, external systems	§1 Architecture	1.10	Named systems and actors. No technology, no internals.

C4 level	Question it answers	Part	Section	Evidence grain required
C2 — Container	Separately deployable or executable units, and data stores	§1 Architecture	1.10	Name, type, technology, one-line responsibility.
C2 — Dynamic	Order of interaction across containers during a cycle	§1 Architecture	1.10	One numbered step per stage ID in §1.2.1.
C2 — Deployment	Which containers run on which nodes	§1 Architecture	1.10	Node names and release lanes. Host detail sits in §2.5.1.
C3 — Component	What sits inside a container and what each part does	§2 Engineering	owning layer	Every component resolves to repo : path, or it is not a component.
C3 — Dynamic	Component interaction within a single stage	§2 Engineering	2.2.4	Steps resolve to file : line.
C4 — Code	What the code does on the write path	§2 Engineering	2.2.5	file : line at a pinned commit. A table, not a diagram, by default.

The dividing line is not diagram complexity, it is whether the reader can check the claim. A C2 container can be described from an interview; a C3 component cannot be drawn honestly without opening the repository. That is why the split falls between L2 and L3.

0.10.2 Working order

Do these in sequence. Working out of order is the most common way a C4 set ends up internally inconsistent.

- *Draw C1 first and settle the system boundary in §1.8.1 before anything else. Every later level inherits that boundary, and changing it afterwards invalidates the whole set.*
- *Draw C2. Then stop and check: every container on the diagram must be reachable in a §2 inventory row through its C4 ID. If it is not, either the container is imaginary or the inventory is incomplete — resolve which before continuing.*
- *Decide, per container, whether C3 is warranted. The default is no. It is warranted when the container carries material capitation logic, is the locus of a known defect, or is a decomposition candidate for a follow-on SOW.*
- *Draw each warranted C3 in the engineering section that owns that container (§0.10.3), never in §1. A C3 diagram in the architecture part is detail leaking upward and will push the 30/70 split out of shape.*
- *Draw C4 only where a specific defect argument needs a picture, and name the defect. Otherwise the §2.2.5 control-flow table is the artefact — it carries the same information in a form citable line by line in a finding.*
- *Record the architectural conclusion of the C3 work — what can be decomposed and what blocks it — in §1.3.4. The verdict belongs in §1; the component views that support it stay in §2.*
- *Re-run the view consistency check in §1.8.4 and the cross-notation check in Appendix B.4 before every issue, not once at the end.*

0.10.3 Which engineering section owns which C3

A C3 diagram belongs to the layer that owns its parent container. If two layers both claim a container, the container is doing two jobs and that is itself worth recording in §1.7.

Container type	Owning section	What the C3 view shows
Application, service, executable, scheduled package	§2.2	Classes, modules, procedures, entry points, configuration readers
Integration adapter, endpoint, mapping engine	§2.3	Parsers, mappers, transport handlers, acknowledgement processors
Data store (database)	§2.4	Schemas, stored procedures, ETL packages, views, jobs owned by the store
Scheduler, agent, monitoring or backup component	§2.5	Job definitions, agent processes, service accounts they run under
Cross-cutting concerns	Appendix C.2 / C.4 (Wave 2)	No containers of their own. These sections annotate existing views rather than producing new ones.

0.10.4 The traceability chain

This is the chain the whole document is built to support. A break anywhere in it is a gap to be recorded, not a formatting problem to be tidied.

Step	Artefact	Lives in
1	External system or actor on the C1 view	§1.8
2	Container it talks to (CAP-C4-CON-nn)	§1.8
3	Inventory row carrying that C4 ID	§2.2.1 / §2.3.1 / §2.4.1 / §2.5.1
4	Component inside it (CAP-C4-CMP-nn)	C3 view in the owning engineering section
5	Source location: repo@commit : file : line	§2.1.4 and the owning inventory
6	Step on the write path that executes it	§2.2.5
7	Register ID for any defect found there	§3.1

0.10.5 Rules and common failure modes

- *Element names are identical across C1, C2, C3 and every §2 inventory row. A container renamed between levels breaks traceability silently and nothing will flag it.*
- *A container that cannot be decomposed — vendor binary, no source, no symbols — is recorded as NOT DECOMPOSED with the reason. Never omitted, because an absent C3 reads as "nothing interesting inside".*
- *A C3 component with no repo : path is not a component, it is a guess. Remove it or tag it INFERRED and raise a CQ.*
- *No components on a C2 diagram and no infrastructure on a C1. Mixed abstraction is the failure C4 exists to prevent.*
- *If the C2 view needs more than roughly twenty containers, the boundary in §1.8.1 is drawn in the wrong place. Fix the boundary, not the diagram.*
- *Where a shared database is used as an integration path between otherwise separate applications, it is a container and that coupling appears as relationships between containers. Hiding it inside a boundary makes the platform look decomposable when it is not.*
- *The container-grain dynamic view has one step per stage in §1.2.1. Component-grain sequences are separate views and live in §2.2.4.*
- *Do not draw C3 for every container. A C4 set that nobody maintains is worse than one that stops at C2 and says so.*

0.10.6 Decomposition tracker

One row per container on the C2 view. This table is how a reviewer checks that the C3 layer is deliberate rather than partial.

C4 container ID	Container	C3 drawn?	Owning §	Reason if not drawn	Tag
CAP-C4-CON-01		No		Vendor binary, no source available	

C4 container ID	Container	C3 drawn?	Owning §	Reason if not drawn	Tag

0.11 Delivery waves

This document is populated in two waves. Wave 1 carries the sections needed to describe the capitation path end to end and to support the follow-on scoping decisions; it is the publishable unit. Wave 2 is deferred to Appendix C and completed afterwards. Deferred does not mean optional — a Wave 1 issue states plainly, in §0.8 and on the appendix face, that Wave 2 is outstanding.

Wave	Sections	Why
Wave 1	§1.1 Scope and boundary	Everything downstream inherits the boundary; nothing else can be completed until it is settled
Wave 1	§1.2 / §2 stage detail — end-to-end architecture	The process spine the rest of the document hangs from
Wave 1	§1.3 ↔ §2.2 Application	Where the write-path defects live
Wave 1	§1.4 ↔ §2.3 Integration	Carries the managed-care lane, previously under-represented
Wave 1	§1.5 ↔ §2.4 Data	The decomposition blocker and the reconciliation machinery
Wave 1	§1.6 ↔ §2.5 Infrastructure and operations	Release lanes, scheduling and the runbook as executed
Wave 1	§1.7 Constraints · §1.8 Views · §2.1 Conventions	Enablers. Without §2.1 no Wave 1 section can carry a citation; without §1.8 the C4 set has no anchor
Wave 1	§3 Findings · §4 Open questions · §5 Traceability	The output of Wave 1, not a separate exercise
Wave 2	Appendix C.1 ↔ C.2 Performance	Needs instrumentation and a measured cycle; cannot be completed from a static read
Wave 2	Appendix C.3 ↔ C.4 Security	Needs access and account-level evidence beyond the Wave 1 routing

Wave 1 target date	
Wave 2 target date	
Wave 1 sections complete at this revision	
Known Wave 1 gaps carried into issue	

*A Wave 1 finding that depends on Wave 2 evidence is tagged **INFERRED** and raised as a CQ in §4 — not held back, and not stated as **CONFIRMED** on partial evidence.*

1. Architecture

Structure, boundaries, responsibilities and coupling. Target roughly 30% of written volume. Every claim made here is evidenced in the matching engineering section in §2 — this part asserts, §2 proves.

1.1 Scope and system boundary

Define the capitation boundary precisely: which payment types, which populations, which release lane, which environments, which calendar periods were observed.

Purpose of this document	
System in scope	
Basis for the boundary	
Release lane and environments observed	
Observation period	
Audience	

1.1.1 In scope

#	In-scope element	Basis for inclusion	Tag

1.1.2 Out of scope

Explicit exclusions with reasons. An unstated exclusion reads as an omission during review.

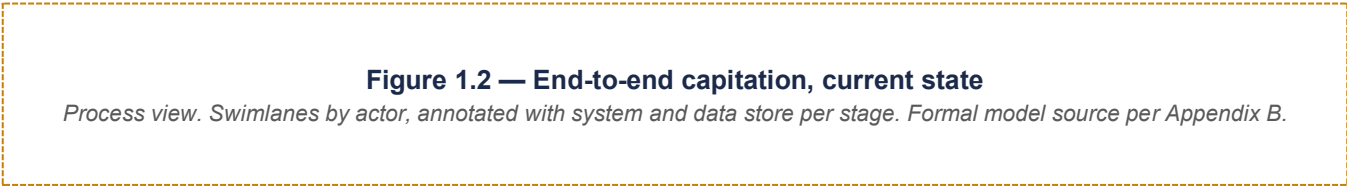
#	Excluded element	Reason	Where it is covered instead

1.1.3 Assumptions

ID	Assumption	Impact if false	Validation route	Tag

1.2 End-to-end capitation architecture

The process spine the rest of the document hangs from. Summary grain only: one row per stage, no implementation detail. Per-stage engineering detail is in §2.2.4.



1.2.1 Stage catalogue

Stage IDs are permanent: CAP-S-01, CAP-S-02 ... Never renumber; append. Every stage appears in exactly one owning engineering section.

Stage ID	Stage	Trigger	Input	Output	Owning layer	Detail in §	Tag
CAP-S-01					Data	2.4	

1.2.2 Actors and responsibilities

Actor / role	Organisation	Responsibility in the capitation path	R / A / C / I

Actor / role	Organisation	Responsibility in the capitation path	R / A / C / I

1.2.3 Cycle types

Cycle type	Stages exercised	Frequency	Differences from the regular cycle	Tag
Regular				

1.3 Application architecture

What executes, what each part is responsible for, and how execution is triggered and sequenced. Component inventories, source paths and configuration keys belong in §2.2, not here.

Figure 1.3 — Application structure
C4 container or component view, per §0.7.

1.3.1 Structure and responsibilities

Element	Type	Responsibility	Depends on	Stages served	Tag

1.3.2 Execution and orchestration model

How execution is triggered	
Sequencing and dependency mechanism	
Concurrency model	
Transaction boundary model	
Failure and restart model	

State held between stages	
---------------------------	--

1.3.3 COTS and customisation boundary

The COTS-versus-custom distinction runs inside the repository, not between repositories. State the boundary at structural grain; the file-grain modification register is §2.2.9.

Element	COTS / Modified COTS / Ohio-original	Basis for classification	Upgrade exposure	Tag

1.3.4 Decomposition posture

The architectural conclusion drawn from the component-level work in §2, without importing its evidence. One row per container on the C2 view. This subsection cannot be completed until the container boundary rule in §0.7 is answered — if a whole database is one container, most rows here will read "not decomposable" for the wrong reason, and the seam will be invisible rather than absent. Component views themselves stay in §2 per §0.10; what belongs here is the verdict and what blocks it.

C4 container ID	Container	Decomposable?	Natural seam	What blocks it today	Feeds which follow-on	Tag
CAP-C4-CON-01		Partially				

Overall posture in one sentence	
Containers that could be carved out with current boundaries	
Containers where the blocker is coupling rather than complexity	
Evidence supporting these verdicts	

"Not decomposable" is a finding and carries a register ID. "Not assessed" is an open question and carries a CQ. Neither is a blank cell.

1.3.5 Architectural decisions as built

Decisions legible in the structure today, whether or not anyone recorded them at the time. State the decision, the constraint that forced it, and what it costs now. This is not a recommendation register.

ID	Decision as built	Constraint that forced it	Consequence today	Evidence in §2	Tag
AD-01					

1.4 Integration architecture

How the capitation path exchanges data with everything outside it. Interface implementation detail, schemas and endpoints are §2.3.

Figure 1.4 — Integration landscape

Show direction, counterparty and mechanism. Do not put endpoints or credentials on this view.

1.4.1 Integration boundaries and counterparties

Counterparty	Type	Direction	Interfaces (ID)	Business criticality	Tag

1.4.2 Integration styles in use

Name every mechanism actually in use — file drop, EDI, web service, message queue, shared database table, manual re-key. Shared database tables and manual steps are integration styles and must be listed as such.

Style	Used for	Interfaces	Coupling introduced	Tag

Style	Used for	Interfaces	Coupling introduced	Tag

1.4.3 Managed-care lane

The managed-care lane carries the majority of capitation traffic and has previously been under-represented in interface inventories. Describe it explicitly rather than folding it into §1.4.1.

Plan / entity	Interfaces used	Payment mechanism	Reconciliation counterparty	Tag

1.4.4 Architectural decisions as built

ID	Decision as built	Constraint that forced it	Consequence today	Evidence in §2	Tag

1.5 Data architecture

Where data lives, what owns it, and how it moves. Table inventories, DDL, row counts and stored-procedure detail are §2.4.

Figure 1.5 — Data architecture

Stores, ownership boundaries and flow direction. Mark shared stores explicitly.

1.5.1 Stores and ownership

Store	Instance	Owns which data	Written by	Read by	Shared?	Tag

1.5.2 Bounded contexts and coupling

Shared SQL Server operating as the de-facto integration bus is the decomposition blocker on this platform. Describe the coupling at structural grain and state whether a bounded context can be drawn at all. Do not soften this.

Candidate bounded contexts identified	
Where context boundaries cannot be drawn, and why	
Cross-context coupling mechanisms in use	
Data ownership disputes or ambiguities	
Consequence for decomposition	

1.5.3 Data flow and lineage overview

Flow	From	To	Mode (batch / real-time)	Grain	Stage	Tag

1.5.4 Reference and rate data model

Reference set	Owner	Effective dating	Version control	Retro handling	Tag

1.5.5 Architectural decisions as built

ID	Decision as built	Constraint that forced it	Consequence today	Evidence in §2	Tag

1.6 Infrastructure architecture

Where the platform runs and how it is operated at structural grain. Host inventories, job definitions, agent coverage and restore evidence are §2.5.



1.6.1 Topology and environments

Environment	Purpose	Release lane	Represents production faithfully?	Tag

1.6.2 Release lanes

Where parallel release stacks exist, state which lane the capitation path executes in today, which components have been cut over, and what still exists only on the older lane. A lane that cannot be switched off is an architectural fact, not an operational detail.

Component / operational layer	Lane A	Lane B	Cutover status	Consequence if the old lane is decommissioned	Tag

Component / operational layer	Lane A	Lane B	Cutover status	Consequence if the old lane is decommissioned	Tag

1.6.3 Availability and recovery posture

Availability approach	
Failover mechanism	
Backup approach	
Stated RPO / RTO	
Whether the posture has been demonstrated	

1.6.4 Platform lifecycle position

End-of-support dates for anything the capitation path depends on. This is architecture because it constrains what can be built on the platform, not merely when to patch.

Element	Version	End of support	Status today	Constraint it imposes	Tag

1.6.5 Architectural decisions as built

ID	Decision as built	Constraint that forced it	Consequence today	Evidence in §2	Tag

1.7 Cross-cutting constraints and characteristics

Constraints that shape every layer, stated once. Repeating them per layer causes them to drift.

1.7.1 Constraints

ID	Constraint	Source	Layers affected	Effect	Tag

1.7.2 Architectural characteristics, as observed

Observed, not required. Where a characteristic cannot be assessed from evidence, record NOT ASSESSED rather than an opinion.

Characteristic	Observed position	Evidence in §2	Tag
<i>Recoverability</i>		§2.5	

1.8 Architecture views (C4 Levels 1–2)

Drawn per §0.7 and populated per §0.10. This section carries C1 System Context, C2 Container, and the container-grain dynamic and deployment views. Component views (C3) and the code-level trace (C4) are evidence and live in §2 — see §0.10.1. Every view carries a version and a source-model reference so it can be regenerated rather than redrawn.

1.8.1 Modelling scope and system boundary

C4 scopes to a software system; capitation is a process crossing several. Either the platform is the system and capitation is traced through it by dynamic views, or the boundary is drawn around the capitation-participating containers only and everything else becomes external. The first is usually more honest on a shared platform; the second draws more cleanly and misleads. State the choice and its consequence.

System in scope for L1	
Basis for the boundary	
What falls outside as an external system	
Consequence of this choice the reader should know about	

1.8.2 View index

View ID	C4 level / type	View name	Question it answers	Source model file	Version	Tag
V-01	L1 System Context					

Index C1 and C2 views only. C3 and C4 views are indexed in the engineering section that owns them.

1.8.3 Views

Figure 1.8.x — [view name]

One figure per view listed in 1.8.2. Caption with view ID, C4 level, model file, version and evidence tag.

At minimum in this section: one L1 System Context, one L2 Container, one container-grain Dynamic view per cycle type in §1.2.3, and one Deployment view. Nothing below container grain is drawn here. Which containers are decomposed to C3, and where those views live, is recorded in the tracker at §0.10.6.

1.8.4 View consistency check

Run before every issue. A failing line means the document is not publishable, not that a note should be added.

#	Check	Result (PASS / FAIL)	Evidence	If FAIL, action
1	Every element on every view appears in the matching §2 inventory			

At minimum: element names identical across C1, C2 and every C3 view in §2; every container reachable in a §2 inventory row through its C4 ID; every container present in the deployment view; every stage in §1.2.1 represented in a container-grain dynamic sequence; every interface in §2.3.1 appearing as a boundary-crossing relationship; no relationship without a stated mechanism; and every container accounted for in the decomposition tracker at §0.10.6. Cross-notation checks against the formal model run separately in Appendix B.4.

2. Engineering

How the architecture in §1 is actually built, at file, object, host and account grain. Target roughly 70% of written volume. Every section here is enumerated deterministically — ripgrep, ScriptDom, Roslyn, SQL queries, directory walks — never estimated and never counted by reading. Each subsection pairs to exactly one architecture layer: 2.2↔1.3, 2.3↔1.4, 2.4↔1.5, 2.5↔1.6. This part also carries the lower half of the C4 set — component views (C3) in the section that owns each container, and the code-level trace (C4) in §2.2.5. Populate them per §0.10.

2.1 Engineering conventions

2.1.1 Code citation standard

Citation format	repo@commit : path/to/file.ext : line or line-range
Revision pinning	Mandatory. Branch-only citations are not CONFIRMED.
Line-range citations	Permitted where a single line is misleading out of context
Excerpt policy	
Where excerpts may not be reproduced	
Verification performed by	

A code citation establishes what the source says, not what the running system does. Both planes must agree before a finding is CONFIRMED — see §0.2 — and where they disagree the runtime plane wins.

2.1.2 Enumeration discipline

Every count in §2 carries the command or query that produced it, so it can be re-run by a reviewer. A count that cannot be reproduced cannot be defended when it is quoted back at you.

Count reported	Section	Command / query	Run by	Date	Tag

2.1.3 Environments and access used

Environment	Accessed for	Access route	Executed by	Attachment Eight position	Tag

Environment	Accessed for	Access route	Executed by	Attachment Eight position	Tag

2.1.4 Repository inventory

Every repository containing code on the capitation path, with the revision pinned. This table is the anchor for every file:line citation in the rest of §2.

Repo	Branch	Commit / tag observed	Date	Primary languages	Build system	Owner	In scope?	Tag

2.2 Application engineering

Evidence for §1.3. What executes, where its source lives, how it is configured, what it does on the write path, and how it behaves when it fails.

2.2.1 Component inventory

Everything that executes: executables, services, packages, agent jobs, stored procedures invoked as entry points, scheduled scripts, desktop tools. Desktop tools especially — they are routinely omitted and routinely load-bearing.

Component ID	C4 element ID	Name	Type	Repo : path	Version	Invoked by	Stage(s)	Tag

Component ID	C4 element ID	Name	Type	Repo : path	Version	Invoked by	Stage(s)	Tag

2.2.2 Entry points and scheduling

Entry point	Type (job / service / manual)	Schedule	Scheduler	Runs as (account)	Downstream chain	Tag

2.2.3 Configuration inventory

Configuration values that change behaviour on this path. Record the logical name and the physical resource it resolves to. Conflating the two has already produced one incorrect finding on this engagement.

Config file : line	Key	Logical value	Resolves to (physical)	Environment	Effect	Tag

Config file : line	Key	Logical value	Resolves to (physical)	Environment	Effect	Tag

2.2.4 Stage execution detail

Copy the block below once per stage in §1.2.1. Keep the field set identical across stages so stages are comparable and gaps show as empty fields rather than as absent sections. Stages owned by another layer still get a block here if application code executes them.

Stage ID and name	
Trigger and schedule	
Preconditions and dependencies	
Inputs (source, format, grain)	
Processing logic	
Executing component (repo@commit : file : line)	
Host and account	
Outputs (target, format, grain)	
Typical duration and volume	
Transaction boundary and commit behaviour	
Restart / rerun behaviour	
Idempotent? (yes / no / evidence)	
Error handling and logging	
Manual intervention required	
Controls and reconciliation points	
Known defects (register ID)	

Evidence tag	

"Idempotent? — no" is a finding, not a field value. Raise it in §3 and cross-reference the register ID here.

2.2.5 Control flow of the voucher / payment write path (C4 Level 4)

The code level of the C4 set, expressed as a table rather than a diagram. Trace the write path end to end at file:line grain: entry → validation → calculation → insert → commit. This is the section a defect argument is built on, so it must be citable line by line. Draw an actual C4 diagram only where a specific defect argument needs a picture, and name that defect in the caption.

[illegible]

2.2.6 Transaction and commit semantics

Distributed transactions, commit-wait behaviour and enlistment are the difference between a slow path and an incorrect one. Where enlistment is inferred from configuration rather than observed at runtime, tag it INFERRED and raise a CQ.

Path	Transaction manager	Enlisted resources	Isolation level	Commit behaviour	Observed or inferred?	Tag

Path	Transaction manager	Enlisted resources	Isolation level	Commit behaviour	Observed or inferred?	Tag

2.2.7 Idempotency and restart behaviour

For each entry point: what happens on re-execution after a partial failure. Restart safety is the field that matters and it is answered with evidence, not with an expectation.

Entry point	Failure point	State left behind	Restart procedure	Restart safe?	Evidence	Tag

2.2.8 Error handling, retry and logging

Component	Error class	Handling	Retry policy	Logged where	Alerting?	Tag

2.2.9 COTS modification register

Vendor files edited in place, at file grain. Distinct from the structural classification in §1.3.3: that says which components are customised, this says which vendor files carry local edits and will therefore conflict at the next vendor upgrade.

Vendor file	Repo : path	Modification	Basis for detecting it	Upgrade conflict risk	Tag

2.2.10 Build, release and artefact provenance

The question here is narrow and important: can you prove the binary running in production was built from the source in §2.1.4? If the answer is no, every code-plane finding in this document is INFERRED against the runtime, however precise its citation.

Build pipeline and where it runs	
Version stamping applied to artefacts	
Deployed artefact version observed in production	
Method used to compare deployed artefact to source	
Result of that comparison	
Components where provenance could not be established	
Consequence for code-plane evidence in this document	

2.2.11 Static analysis coverage and blind spots

State what the analysis could see and, more importantly, what it could not. An analyser blind to a technology on the path leaves a gap closed by runtime evidence or declared open.

Analyser / method	Covers	Cannot see	Path elements left uncovered	How that gap is closed	Tag

2.2.12 Component views (C4 Level 3) — application containers

One view per application container decomposed per §0.10.2. Components here are the units the component inventory in §2.2.1 already lists; this subsection shows how they relate, it does not re-list them. Every component on the view carries a repo : path.

Figure 2.2.12 — Component view (C4 L3): [container name]
Show call direction and, where a relationship crosses a process boundary, the mechanism.

C4 component ID	Component	Parent container	Technology	Responsibility	Repo : path	Tag
CAP-C4-CMP-01						

Containers decomposed at C3 in this section	
Containers deliberately not decomposed	
Reason	

2.2.13 Application defects and gaps

Register ID	Defect	Evidence (file:line / query)	Observed impact	Tag

Register ID	Defect	Evidence (file:line / query)	Observed impact	Tag

2.3 Integration engineering

Evidence for §1.4. Interface by interface, at endpoint, schema and mapping-object grain.

2.3.1 Interface implementation inventory

Use the interface identifiers already issued in the integration crosswalk. Do not mint new IDs; if an interface is missing from the crosswalk, that omission is itself a finding.

[illegible]

2.3.2 Schemas, specifications and companion guides

Interface ID	Transaction set / schema	Version	Specification reference	Mapping owned by	Tag

Interface ID	Transaction set / schema	Version	Specification reference	Mapping owned by	Tag

2.3.3 Mapping and transformation objects

The code that actually performs each translation, cited at file:line or object grain. A mapping maintained in a spreadsheet or a desktop tool is recorded here as such.

Interface ID	Transformation object (repo@commit : file / package)	Technology	Invoked by	Tag

2.3.4 Transfer mechanics and landing zones

Interface ID	Landing path / endpoint	Transfer mechanism	Credential type and location	Retention	Tag

Interface ID	Landing path / endpoint	Transfer mechanism	Credential type and location	Retention	Tag

2.3.5 Acknowledgement, rejection and resubmission

Interface ID	Failure mode	Detection	Acknowledgement path	Resubmission path	Manual?	Tag

2.3.6 Duplicate and replay handling

Per interface: what happens if the same file or message arrives twice. On a payment path this is not a hypothetical.

Interface ID	Duplicate detection mechanism	Behaviour on replay	Idempotent?	Evidence	Tag

2.3.7 Cut-off enforcement and batching

Interface ID	Cut-off	Enforced how (hard / soft / none)	Batch size	Late-arrival handling	Tag

Interface ID	Cut-off	Enforced how (hard / soft / none)	Batch size	Late-arrival handling	Tag

2.3.8 Integration verification evidence

What proves each interface currently works: a monitored transaction, a reconciliation, an acknowledgement file, or nothing. "Nothing" is a legitimate and important answer.

Interface ID	Verification in place	Evidence produced	Frequency	Gap	Tag

2.3.9 Component views (C4 Level 3) — integration containers

One view per integration container decomposed per §0.10.2. Show parsers, mappers, transport handlers and acknowledgement processors, and where each is invoked from. A mapping maintained in a spreadsheet or desktop tool appears here as a component like any other.

Figure 2.3.9 — Component view (C4 L3): [container name]

Mark every component that performs a translation, and name the specification it implements.

C4 component ID	Component	Parent container	Technology	Responsibility	Repo : path	Tag
CAP-C4-CMP-01						

C4 component ID	Component	Parent container	Technology	Responsibility	Repo : path	Tag

Containers decomposed at C3 in this section	
Containers deliberately not decomposed	
Reason	

2.3.10 Integration defects and gaps

Register ID	Defect	Interface ID	Evidence	Observed impact	Tag

2.4 Data engineering

Evidence for §1.5. Physical schema, database code, lineage, and the control and reconciliation machinery that sits on top of it.

2.4.1 Instances and databases

Instance / host	Database	C4 container ID	Role (read / write / both)	Version	Release lane	Tag

Instance / host	Database	C4 container ID	Role (read / write / both)	Version	Release lane	Tag

2.4.2 Table inventory

Grain and key are the two fields that determine whether reconciliation is possible at all. A keyless table on a financial path is a finding in its own right.

Table	Database	Grain	Primary key	Row count	Growth / cycle	Read / written by	Tag

2.4.3 Database code objects

Business logic in the data tier is the decomposition blocker on this platform, so it is inventoried as code rather than as schema. Guarded means the write path validates before it writes and behaves correctly on re-execution; record the basis for that judgement, not just the verdict.

[illegible]

2.4.4 Cross-database and linked-server dependencies

Every cross-database reference on the capitation path is a decomposition blocker and must be enumerated, not sampled. Record the enumeration query in §2.1.2.

[illegible]

From (db.object)	To (db.object)	Mechanism (3-part / linked server / MSDTC)	Direction	Stage	Tag

2.4.5 ETL and package inventory

Package / job	Technology	Repo : path	Source	Target	Schedule	Tag

2.4.6 Lineage

Source → transform → target for every hop on the path, including file-based hops and any hop through a spreadsheet or desktop application.

Hop	Source	Transform (object)	Target	Mode	Frequency	Tag

2.4.7 Reference and rate data loading

Reference set	Location	Loaded by (object)	Effective dating	Retro replay mechanism	Tag

2.4.8 Indexing, contention and physical design

Only where it bears on correctness or on the batch window. Blocking and lock escalation on the write path belong here; general index hygiene does not.

Object	Physical design note	Contention observed	Evidence (query / wait stats)	Tag

2.4.9 Data quality controls

Control	Stage	What it checks	Enforcement (hard stop / warning / none)	Where results go	Tag

2.4.10 Reconciliation and control engineering

What is reconciled to what, by which object, at what frequency, to what tolerance, and where the result is recorded. A reconciliation with no recorded tolerance is not a control.

Recon ID	Source A	Source B	Performed by (object / person)	Frequency	Tolerance	Result recorded where	Tag

Then state what is not reconciled. Say plainly that it is not reconciled and what the exposure is. Do not soften this table.

Gap	Between what and what	Exposure	Why not reconciled today	Tag

2.4.11 Financial posting and downstream reporting

Posting	Triggered by (object / stage)	Target ledger / system	Reversal path	Federal reporting impact	Tag

2.4.12 Retention, history and archival

Data set	Retention period	Archival mechanism	Restorable?	Basis (contract / policy)	Tag

2.4.13 Component views (C4 Level 3) — data containers

One view per database decomposed per §0.10.2. Components inside a data store are schemas, stored-procedure groupings, ETL packages and views. This is the view that makes business logic held in the data tier visible as structure rather than as a list of object names.

Figure 2.4.13 — Component view (C4 L3): [container name]
Group by responsibility, not by object type. Mark every grouping that writes on the payment path.

C4 component ID	Component	Parent container	Technology	Responsibility	Repo : path	Tag
CAP-C4-CMP-01						

Containers decomposed at C3 in this section	
Containers deliberately not decomposed	
Reason	

2.4.14 Data defects and gaps

Register ID	Defect	Object	Evidence	Observed impact	Tag

2.5 Infrastructure and operations engineering

Evidence for §1.6. Hosts, jobs, agents, backups and the runbook as operators actually execute it.

2.5.1 Host inventory

Only hosts on the capitation path. Record the release lane explicitly — a host inventory that does not distinguish parallel release stacks will double-count and mislead.

Hostname	C4 node ID	Role	Release lane	OS / version	Spec	Environment tag	Tag

2.5.2 Deployment mapping

Container / component	Host	Path / service name	Runs as	Instances	Tag

Container / component	Host	Path / service name	Runs as	Instances	Tag

2.5.3 Job scheduling and orchestration

Job name	Scheduler	Host	Schedule	Dependencies	Failure action	Tag

2.5.4 Observability and agent coverage

State coverage and the gaps together. A monitoring tool that cannot see a given execution technology is a blind spot on the path and must be recorded as one.

Component / host	APM agent	Log agent	Alerting	Coverage gap	Tag

Component / host	APM agent	Log agent	Alerting	Coverage gap	Tag

2.5.5 Backup, availability and recovery configuration

Asset	Backup method	Frequency	Last successful run	Last verified restore	HA configuration	Tag

2.5.6 Lifecycle and end-of-support position

Element	Version	End of support	Status today	Exposure to this path	Tag

2.5.7 Run procedure

The runbook as operators actually execute it, not as documented. Where the two differ, record both and tag the difference.

Step	Action	Performed by	Automated / manual	Verification	Elapsed	Tag

Step	Action	Performed by	Automated / manual	Verification	Elapsed	Tag

2.5.8 Manual touchpoints

Count them. A manual-step count is the single most quotable figure a current-state assessment produces, and it must be defensible step by step.

#	Manual step	Stage	Performed by	Frequency	Duration	Failure exposure	Tag

2.5.9 Failure modes and restart procedures

Failure mode	Observed symptom	State left behind	Restart procedure	Restart safe?	Tag

2.5.10 Incident history

Date	Incident ref	Description	Root cause	Cycle impact	Resolution	Tag

2.5.11 Support model

Primary support team	
Escalation path	
On-call coverage for a cycle run	
Vendor dependencies	
Documented runbook location	
Single points of human knowledge	

“Single points of human knowledge” means named individuals whose absence would stop or endanger a cycle. Name them. This is uncomfortable to write and it is the most valuable row in the section.

2.5.12 Component views (C4 Level 3) — operational containers

One view per operational container decomposed per §0.10.2 — schedulers, agents, backup and monitoring components. Draw these only where the operational tooling is itself part of the finding, for example where a component exists on one release lane and not the other.

Figure 2.5.12 — Component view (C4 L3): [container name]

Show which node each component runs on and which account it runs under.

C4 component ID	Component	Parent container	Technology	Responsibility	Repo : path	Tag
CAP-C4-CMP-01						

Containers decomposed at C3 in this section	
Containers deliberately not decomposed	
Reason	

2.5.13 Infrastructure defects and gaps

Register ID	Defect	Host / component	Evidence	Observed impact	Tag

3. Consolidated findings, defects and risks

Findings recorded here must exist in the engagement register and carry its identifier. This document does not mint findings; it evidences them. If something new emerges while writing, add it to the register first, then cite it here. Every row links back to the layer subsection that evidences it.

3.1 Confirmed defects

Register ID	Layer	Defect	Evidenced in §	Observed impact	Tag

3.2 Architectural constraints and structural issues

Register ID	Constraint	Why it is structural rather than remediable in place	Evidenced in §	Tag

3.3 Risks

Risk ID	Risk	Trigger / forcing event	Date	Likelihood	Impact	Current mitigation

Risk ID	Risk	Trigger / forcing event	Date	Likelihood	Impact	Current mitigation

3.4 Retracted and refuted items

Retractions are recorded, never deleted. State what was asserted, what disproved it, and when.

Item	Originally asserted	Disproving evidence	Date retracted	Tag

4. Open questions

Every SME GATE and INFERRED tag in this document must appear here as a numbered question with a named owner. A document whose open-question list is shorter than its INFERRED tag count is internally inconsistent.

CQ ID	Question	Section	Plane	Owner	Raised	Due	Status	Answer / disposition

5. Traceability

Links this document to the engagement register. Identifiers follow the established convention: PS-*nn* problem statement, RC-*nn* root cause, F-*nnn* finding, REC-*nn* recommendation, WO-*n.n* work order. Recommendations and work orders are referenced only — they are not authored here.

Architecture §	Engineering §	Stage ID	C4 element / model ref	Register link (PS / RC / F)	Referenced REC / WO	Tag

Appendix A — Business context

Retained as an appendix. The body of this document is technical architecture and engineering only; the material here is the operating context the technical sections are measured against. Where a technical section depends on a figure from this appendix it cites the row rather than restating it.

A.1 What capitation means on this platform

Plain-language definition as the platform implements it, not the textbook definition. Include the unit of payment, the entity paid, the period covered, and how retroactivity is treated.

A.2 Business capabilities exercised

Map to MITA business areas and to the capability taxonomy already in use on the engagement. Do not invent a new taxonomy.

Capability ID	Capability	MITA area / maturity	Systems realising it	Tag

A.3 Contractual, SLA and regulatory drivers

Cite the instrument and clause. Where an SLA is analytically mapped onto this path rather than contractually stated for it, say so on the face of the row — an inherited SLA presented as a contractual one is a material misstatement.

Ref	Instrument and clause	Obligation	Applies to which stage	Contractual or derived?	Tag

A.4 Payment calendar and cycle

The actual operating calendar: cycle frequency, cut-off, run window, approval gates, disbursement date, and what happens when a run slips. Include the off-cycle and adjustment runs.

Cycle	Frequency	Cut-off	Run window	Approval gate(s)	Disbursement	Tag

Cycle	Frequency	Cut-off	Run window	Approval gate(s)	Disbursement	Tag

A.5 Volumes and materiality

Business-level figures with the period they were measured over. Engineering measurements of the same quantities live in Appendix C.2.2 and must agree with these or the disagreement is recorded.

Measure	Value	Period observed	Source	Tag

A.6 Actors and responsibilities

Actor / role	Organisation	Responsibility in the capitation path	R / A / C / I

Appendix B — ArchiMate 4.0 model

Retained as an appendix. The body communicates in C4; this appendix holds the formal model those views are generated from, the mapping between the two notations, and the cross-notation consistency check.

B.1 Standard and authority

Standard	<i>ArchiMate 4.0 (C260)</i>
Superseded versions	<i>ArchiMate 3.2 is not used or referenced anywhere in this document</i>
Authoritative model	
Derived artefacts	
Model file location and version control	
Authoring tool	
Regeneration procedure for the C4 views in §1.8	
Consistency check performed at	

B.2 Element mapping

Every C4 element must be traceable to an ArchiMate element or explicitly recorded as unmapped. Unmapped elements are where the two models drift.

C4 element	ArchiMate 4.0 equivalent	Note
Person	Business Actor / Business Role	Role where it is a job function, Actor where named
Software System	Application Component (coarse-grained)	Or Application Collaboration where it is a set of cooperating parts
Container (application)	Application Component	One level of nesting below the system
Container (data store)	Node + System Software + Artifact / Data Object	A database is one C4 container but three ArchiMate elements — record all three
Component	Application Component (nested) or Application Function	Choose one and use it consistently throughout
Code element	Not modelled	Cite file:line in §2.2.5 instead
Relationship	Serving / Flow / Access / Triggering	C4 arrows are untyped; the ArchiMate relationship type must be stated
Deployment node	Node / Device / Technology Service	Maps to the host inventory in §2.5.1

B.3 Layer views

Layer views are drawn only where they carry information the C4 set does not. Duplicating the container view as an application-layer diagram adds maintenance burden and no reader value.

View ID	Layer	Viewpoint	Question it answers	Not duplicated by which C4 view	Version	Tag

View ID	Layer	Viewpoint	Question it answers	Not duplicated by which C4 view	Version	Tag

Figure B.3 — [layer view name]

One figure per view listed in B.3. Caption with view ID, layer, viewpoint, model file, version, evidence tag.

B.4 Cross-notation consistency check

Run before every issue, alongside the view check in §1.8.4. A failing line means the document is not publishable.

#	Check	Result (PASS / FAIL)	Evidence	If FAIL, action
1	Every C4 element in §1.8 maps to a model element per B.2, or is recorded as unmapped			

B.5 Enterprise corpus alignment

Elements that also appear in issued enterprise instruments must carry the same identifier and the same meaning. Where they diverge, the divergence is a finding rather than a modelling preference.

Element	Identifier here	Enterprise instrument	Identifier there	Agree?	If not, disposition

Appendix C — Wave 2: Performance and Security

Deferred to the second wave per §0.11. These are full layers with the same architecture-and-engineering pairing as the body — C.1 pairs to C.2, C.3 pairs to C.4 — held here because both require evidence the Wave 1 collection does not produce: an instrumented cycle for performance, and account-level access for security. They are placed in an appendix to keep the Wave 1 body publishable, not because they are lower value. When Wave 2 completes, either promote these into §1 and §2 as layers 1.7/2.6 and 1.8/2.7, or leave them here and say so on the cover — do not leave the question open.

Wave 2 status	
Blocked on	
Owner	
Target date	
Wave 1 findings currently tagged INFERRED pending this work	

C.1 Performance architecture

The structural properties that determine how the path performs: where work queues, what serialises, what the batch window is, and where headroom exists. Measurements belong in Appendix C.2 — this section is about why the numbers come out the way they do.

Figure C.1 — Workload and critical path

Stage timing over the cycle window, with the critical path marked.

C.1.1 Workload model

Workload	Arrival pattern	Volume driver	Concurrency	Peak condition	Tag

C.1.2 Scaling and serialisation points

Name what scales and what does not. A single-threaded step, a table-level lock, or a step that must complete before any other can start are the properties that set the ceiling.

Element	Scales how (out / up / not at all)	Serialisation point?	Ceiling imposed	Tag

Element	Scales how (out / up / not at all)	Serialisation point?	Ceiling imposed	Tag

C.1.3 Batch window and timing constraints

Cycle window available	
Critical path through the window	
Slack at current volumes	
What happens when the window is missed	
Downstream dependencies on completion time	

C.1.4 Performance targets applying to this path

Cite the instrument and clause. Where a target is analytically mapped onto this path rather than contractually stated for it, say so on the face of the row — an inherited SLA presented as a contractual one is a material misstatement. Targets themselves live in Appendix A.3.

Ref	Target	Applies to which stage	Contractual or derived?	Measured in Appendix C.2	Tag

C.1.5 Architectural decisions as built

ID	Decision as built	Constraint that forced it	Consequence today	Evidence in §2	Tag

C.2 Performance engineering

Evidence for Appendix C.1. Measurements with a stated method and observation window. Where a measurement could not be taken, record NOT MEASURED and state what instrumentation would be needed — an unmeasured claim is worse than an acknowledged gap.

C.2.1 Measurement method and instrumentation

Measure	Instrument / query used	Granularity	Observation window	Limitations	Tag

C.2.2 Observed volumes

Figures with the period measured and the query behind them. Do not carry volumes forward from Appendix A or from other instruments without re-citing the source.

Measure	Value	Period observed	Source / query	Tag
<i>Member months per cycle</i>				CONFIRMED

C.2.3 Stage-level performance baseline

Stage ID	Elapsed	Rows processed	Throughput	Resource profile	Measured how	Tag

Stage ID	Elapsed	Rows processed	Throughput	Resource profile	Measured how	Tag

C.2.4 Bottleneck analysis

Where the time actually goes. Wait statistics, blocking chains, plan-level findings, IO profiles. An assertion that something is "slow because of volume" needs the failure signature to support it — on this engagement that argument has already failed once against the evidence.

Stage / object	Bottleneck	Evidence (wait stats / plan / trace)	Contribution to elapsed	Tag

C.2.5 Query and plan-level findings

Query / object	Issue	Plan evidence	Rows / duration	Tag

Query / object	Issue	Plan evidence	Rows / duration	Tag

C.2.6 Critical path and window utilisation

Stage ID	Planned start	Observed duration	Slack	On critical path?	Constraint

C.2.7 Capacity and scaling limits

Resource	Current utilisation	Headroom	Binding constraint	What happens at the limit	Tag

C.2.8 Load and stress test evidence

If no load testing exists for this path, say so in one line and record the consequence. That absence is itself a finding.

Load testing performed?	
Scenarios covered	
Peak volume tested against production peak	
Date last executed	

Consequence of any gap	
------------------------	--

C.2.9 Position against stated targets

Target ref (Appendix C.1.4)	Target	Observed	Meets?	Basis of comparison	Tag

C.2.10 Performance defects and gaps

Register ID	Defect	Stage / object	Evidence	Observed impact	Tag

C.3 Security architecture

Trust boundaries, identity model, and where protected data sits. Account names, key locations, patch levels and audit-log implementation are Appendix C.4.

Figure C.3 — Trust boundaries and data classification

Mark every boundary crossing carrying PHI.

C.3.1 Trust boundaries

Boundary	Between what and what	Crossed by	Control at the boundary	PHI crosses?	Tag

Boundary	Between what and what	Crossed by	Control at the boundary	PHI crosses?	Tag

C.3.2 Identity and authorisation model

Identity sources in use	
Authorisation model	
Service-to-service authentication approach	
Privileged access model	
Segregation of duties on the payment path	

C.3.3 Data classification and PHI exposure

Data element / store	Classification	PHI / PII	Exposed at which stage	Protection approach	Tag

C.3.4 Access-model constraints

How Attachment Eight and any other access restriction shape who can operate, observe or troubleshoot this path — and what that means for run-time support coverage. This is an architectural constraint, not just a process rule.

Restriction	
Who may execute on production	
Effect on support coverage	
Effect on the evidence in this document	

C.3.5 Architectural decisions as built

ID	Decision as built	Constraint that forced it	Consequence today	Evidence in §2	Tag

ID	Decision as built	Constraint that forced it	Consequence today	Evidence in §2	Tag

C.4 Security engineering

Evidence for Appendix C.3. Accounts, credentials, cipher configuration, audit implementation and patch position, at named-artefact grain. Do not reproduce secrets or their values in this document — record location and type only.

C.4.1 Identity and service account inventory

Account	Type	Used by (component)	Privilege level	Rotation	Interactive logon?	Tag

C.4.2 Authentication and authorisation implementation

Component / interface	AuthN mechanism	AuthZ mechanism	Enforced where (file:line / config)	Tag

C.4.3 Secrets and credential handling

Where credentials actually live, not where policy says they should. Connection strings in configuration files, embedded credentials and shared accounts are recorded here as found.

Credential	Stored where	Protection (encrypted / plaintext / vault)	Used by	Rotation evidence	Tag

C.4.4 Encryption in transit and at rest

Path / store	In transit	Protocol / cipher and version	At rest	Mechanism	Tag

C.4.5 PHI data flow and exposure points

Flow / store	PHI elements present	Exposure point	Control applied	Residual exposure	Tag

C.4.6 Audit logging implementation

Event type	Captured by (object)	Written where	Retained for	Sufficient for audit?	Gap	Tag

C.4.7 Access enforcement and segregation of duties

Control	Enforced how	Applies to	Bypass possible?	Evidence	Tag

C.4.8 Patch and vulnerability position

Component	Version	Patch level observed	Known exposure	Compensating control	Tag

C.4.9 Security defects and gaps

Where a security finding is sensitive, record the finding and its register ID here and hold the detail in a restricted annex. Do not omit it silently.

Register ID	Finding	Evidence	Observed exposure	Detail held where	Tag

Appendix D — Evidence log

Every artefact relied on, at document grain. A claim whose evidence is not logged here is not **CONFIRMED**, whatever the tag says.

Evidence ID	Type (code / query / doc / interview)	Reference	Collected by	Date	Supports §

Appendix E — Source artefact inventory

Artefact	Location / repository	Commit / version	Retrieved	Used in §

Appendix F — Interview and walkthrough log

Walkthroughs are evidence. Record who said what, when, and whether it was subsequently corroborated in code or runtime. Uncorroborated statements remain SME GATE.

Date	Participants	Organisation	Topic	Key statements	Corroborated?	Supports §

Appendix G — Glossary and abbreviations

Term	Expansion	Meaning on this platform

Appendix H — Review and sign-off

Reviewer	Organisation	Role	Sections reviewed	Comments resolved	Date	Signature

Sign-off records that the current state as described has been reviewed and is not disputed. It does not signify agreement with any conclusion drawn from it elsewhere.